

Компоненты React

Введение в компоненты React

Компоненты являются основными строительными блоками любого React-приложения. Компонент - это изолированная часть пользовательского интерфейса, которая может быть использована повторно.

Функциональные компоненты

Функциональные компоненты - это JavaScript-функции, которые возвращают JSX (React-элементы).

```
function Welcome(props) {  
  return <h1>Привет, {props.name}</h1>;  
}
```

Функциональные компоненты стали предпочтительным способом создания компонентов в React с введением хуков в React 16.8.

Классовые компоненты

Классовые компоненты - это JavaScript-классы, которые расширяют `React.Component` и имеют метод `render()`.

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Привет, {this.props.name}</h1>;  
  }  
}
```

Props (свойства)

Props (свойства) - это входные данные компонентов React. Они передаются от родительского компонента к дочернему.

```
function Welcome(props) {  
  return <h1>Привет, {props.name}</h1>;  
}  
  
// Использование компонента с props  
<Welcome name="Алиса" />
```

Props являются только для чтения и не должны изменяться внутри компонента.

Композиция компонентов

Компоненты могут ссылаться на другие компоненты в своем выводе. Это позволяет использовать одни и те же компоненты на разных уровнях абстракции.

```
function App() {  
  return (  
    <div>  
      <Welcome name="Алиса" />  
      <Welcome name="Боб" />  
      <Welcome name="Чарли" />  
    </div>  
  );  
}
```

Фрагменты

Фрагменты позволяют группировать список дочерних элементов без добавления дополнительных узлов в DOM.

```
function App() {  
  return (  
    <React.Fragment>  
      <h1>Заголовок</h1>  
      <p>Параграф</p>  
    </React.Fragment>  
  );  
}  
  
// Сокращенный синтаксис  
function App() {  
  return (  
    <>  
      <h1>Заголовок</h1>  
      <p>Параграф</p>  
    </>  
  );  
}
```

Условный рендеринг

В React можно создавать компоненты, которые отображают различный контент в зависимости от состояния.

```
function UserGreeting(props) {
  return <h1>С возвращением!</h1>;
}

function GuestGreeting(props) {
  return <h1>Пожалуйста, зарегистрируйтесь.</h1>;
}

function Greeting(props) {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
    return <UserGreeting />;
  }
  return <GuestGreeting />;
}
```

Списки и ключи

Для отображения списков элементов в React используется метод `map()`. Каждому элементу списка необходимо присвоить уникальный ключ.

```
function NumberList(props) {
  const numbers = props.numbers;
  const listItems = numbers.map((number) =>
    <li key={number.toString()}>
      {number}
    </li>
  );
  return (
    <ul>{listItems}</ul>
  );
}
```

Ключи помогают React идентифицировать, какие элементы были изменены, добавлены или удалены.

Обработка событий

Обработка событий в React-элементах очень похожа на обработку событий в DOM-элементах, но с некоторыми синтаксическими отличиями.

```
function Button() {
  function handleClick() {
    alert('Кнопка была нажата!');
  }

  return (
```

```

    <button onClick={handleClick}>
      Нажми меня
    </button>
  );
}

```

Подъем состояния

Часто несколько компонентов должны отражать одни и те же изменяющиеся данные. Рекомендуется поднимать общее состояние до ближайшего общего предка.

```

function TemperatureInput(props) {
  return (
    <fieldset>
      <legend>Введите температуру в {props.scale}</legend>
      <input
        value={props.temperature}
        onChange={(e) => props.onTemperatureChange(e.target.value)} />
    </fieldset>
  );
}

function Calculator() {
  const [temperature, setTemperature] = useState('');
  const [scale, setScale] = useState('c');

  function handleCelsiusChange(temperature) {
    setScale('c');
    setTemperature(temperature);
  }

  function handleFahrenheitChange(temperature) {
    setScale('f');
    setTemperature(temperature);
  }

  const celsius = scale === 'f' ? tryConvert(temperature, toCelsius) :
temperature;
  const fahrenheit = scale === 'c' ? tryConvert(temperature, toFahrenheit) :
temperature;

  return (
    <div>
      <TemperatureInput
        scale="c"
        temperature={celsius}
        onTemperatureChange={handleCelsiusChange} />
      <TemperatureInput
        scale="f"
        temperature={fahrenheit}

```

```
        onTemperatureChange={handleFahrenheitChange} />  
      </div>  
    );  
  }
```

Заключение

Компоненты - это основа React. Понимание того, как создавать и использовать компоненты, является ключевым для разработки эффективных React-приложений. Функциональные компоненты с хуками стали современным стандартом, но также важно понимать классовые компоненты, особенно при работе с существующим кодом.